

CI and CD

সফটওয়্যার ইঞ্জিনিয়ারিংয়ে **CI/CD** হলো এমন একটি আধুনিক পদ্ধতি যা কোড লেখা থেকে শুরু করে সেটি গ্রাহকের কাছে পৌঁছে দেওয়া পর্যন্ত পুরো প্রক্রিয়াটিকে একটি "অটোমেটেড পাইপলাইন"-এ নিয়ে আসে।

সহজভাবে বলতে গেলে, আগে ডেভেলপাররা মাস শেষে সব কোড একসাথে জমা দিতেন এবং তারপর পরীক্ষা চলত। এখন CI/CD-এর মাধ্যমে সেটি প্রতিদিন, এমনকি প্রতি ঘণ্টায় স্বয়ংক্রিয়ভাবে করা হয়।

১. CI: Continuous Integration (ক্রমাগত একীকরণ)

CI হলো এমন একটি প্র্যাকটিস যেখানে ডেভেলপাররা তাদের কোড বারবার (দিনে কয়েকবার) একটি কমন ব্রাঞ্চে (যেমন: GitHub/GitLab-এর Main branch) জমা দেয়।

- **কীভাবে কাজ করে?** যখনই কোনো ডেভেলপার কোড পুশ (Push) করে, সাথে সাথে একটি অটোমেটেড সিস্টেম কোডটি নিয়ে "Build" করে এবং সব "Tests" রান করে।
- **কেন এটি গুরুত্বপূর্ণ?** এতে করে কোডে কোনো ভুল থাকলে বা অন্য কারো কোডের সাথে সংঘর্ষ (Conflict) হলে তা শুরুতেই ধরা পড়ে।
- **উদাহরণ:** ধরুন আপনি একটি নতুন বাটন যোগ করলেন। CI সিস্টেম সাথে সাথে চেক করবে ওই বাটন যোগ করার ফলে অন্য কোনো ফিচার ভেঙে গেছে কি না।

২. CD: এর দুটি মানে হয়

CD আসলে দুটি ধাপে কাজ করে: **Continuous Delivery** এবং **Continuous Deployment**।

ক. Continuous Delivery (ক্রমাগত সরবরাহ)

এটি CI-এর পরের ধাপ। এখানে কোড টেস্ট হওয়ার পর স্বয়ংক্রিয়ভাবে একটি "প্যাকেজ" বা "আর্টিফ্যাক্ট" (যেমন: .exe বা Docker Image) তৈরি হয় যা যেকোনো সময় সার্ভারে দেওয়ার জন্য প্রস্তুত থাকে।

- **বৈশিষ্ট্য:** কোডটি প্রোডাকশনে (লাইভ সার্ভারে) দেওয়ার জন্য শুধু একজন মানুষের "Manual Approval" বা অনুমতির প্রয়োজন হয়।

খ. Continuous Deployment (ক্রমাগত স্থাপনা)

এটি হলো অটোমেশনের সর্বোচ্চ পর্যায়। এখানে মানুষের কোনো অনুমতির প্রয়োজন হয় না।

- **বৈশিষ্ট্য:** কোড যদি সব টেস্ট পাস করে, তবে সেটি অটোমেটিক সরাসরি লাইভ সার্ভারে চলে যায়। Netflix বা Facebook-এর মতো কোম্পানিগুলো দিনে হাজার হাজার বার এভাবে কোড ডেপ্লয় করে।

৩. CI/CD পাইপলাইনের ধাপসমূহ (The Pipeline)

একটি আদর্শ পাইপলাইন নিচের ধাপগুলো অনুসরণ করে:

1. **Source:** ডেভেলপার কোড লিখে Git-এ পুশ করে।
2. **Build:** কোডটি কম্পাইল করা হয় এবং প্রয়োজনীয় লাইব্রেরি যুক্ত করা হয়।
3. **Test:** অটোমেটেড ইউনিট টেস্ট এবং ইন্টিগ্রেশন টেস্ট রান করা হয়।
4. **Staging:** প্রোডাকশনে দেওয়ার আগে একটি ডামি সার্ভারে কোডটি চালিয়ে দেখা হয়।
5. **Deploy:** সব ঠিক থাকলে আসল সার্ভারে অ্যাপটি লাইভ করা হয়।

৪. জনপ্রিয় CI/CD টুলস

বর্তমান সময়ে এই কাজগুলো করার জন্য নিচের টুলগুলো সবচেয়ে বেশি ব্যবহৃত হয়:

- **GitHub Actions:** বর্তমানে সবচেয়ে জনপ্রিয় এবং সহজ।
- **Jenkins:** অনেক পুরনো এবং অত্যন্ত শক্তিশালী (Self-hosted)।
- **GitLab CI/CD:** গিটল্যাবের ভেতরেই ইন-বিল্ট থাকে।
- **CircleCI / Travis CI:** ক্লাউড-বেসড অটোমেশনের জন্য জনপ্রিয়।

একনজরে পার্থক্য:

বৈশিষ্ট্য	CI	Continuous Delivery	Continuous Deployment
মূল লক্ষ্য	কোড মার্জ ও টেস্ট করা।	প্রোডাকশনের জন্য কোড প্রস্তুত রাখা।	অটোমেটিক গ্রাহকের কাছে পৌঁছে দেওয়া।
মানুষের কাজ	কোড লেখা ও পুশ করা।	রিলিজ বাটনে ক্লিক করা।	কিছুই না (পুরোটা অটো)।

CI/CD কী?

CI/CD = Continuous Integration + Continuous Delivery / Deployment

সহজ কথায় 🙌

কোড লিখলেই automatically **build** → **test** → **deploy** হয়ে যাবে
মানুষে মানুষে “run করো”, “deploy করো” বলা লাগবে না

CI (Continuous Integration)

CI কী?

👉 বারবার ছোট ছোট কোড change **main branch-এ merge** করা

👉 প্রতিবার **auto build + auto test**

CI না থাকলে ❌

- Code merge করলে break হয়
- “আমার মেশিনে কাজ করে” সমস্যা
- Bug late ধরা পড়ে

CI থাকলে ✅

- Early bug detection
- Clean codebase
- Team collaboration smooth

CI Flow

Code → Push → Build → Test → Report

CD (Continuous Delivery vs Deployment)

CD – Continuous Delivery

👉 Code **deploy-ready** থাকে

👉 **Manual approve** দিলে production-এ যায়

Build → Test → Staging → (Manual approve) → Prod

CD – Continuous Deployment

👉 কোনো manual step নাই

👉 Test pass হলেই **auto production deploy**

Build → Test → Prod (Auto)

✂ Delivery vs Deployment

Delivery	Deployment
Manual approval	Fully automatic
Safer	Faster
Enterprises	Startups

🏢 CI/CD Pipeline Architecture

Developer

↓ git push

CI Tool (GitHub Actions / Jenkins)

↓

Build → Test → Security Scan



Docker Image



Deploy (Server / Kubernetes / Cloud)

🔧 Popular CI/CD Tools

CI Tools

- GitHub Actions ★
- GitLab CI
- Jenkins
- CircleCI

CD Tools

- ArgoCD (Kubernetes)
- Spinnaker
- GitHub Actions

🔧 Example: GitHub Actions (CI/CD)

.github/workflows/ci.yml

name: CI Pipeline

on: [push]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Setup Node

uses: actions/setup-node@v4

with:

node-version: 18

- name: Install deps

run: npm install

- name: Run tests

run: npm test

👉 push করলেই:

- code checkout
- install
- test

CD Example (Docker + K8s)

- name: Build Docker image
run: `docker build -t myapp .`
- name: Push image
run: `docker push myrepo/myapp`
- name: Deploy to Kubernetes
run: `kubectl apply -f deployment.yml`

CI/CD + Docker + Kubernetes

Git Push

- CI (Test)
- Docker Build
- Push to Registry
- Kubernetes Deploy
- Service live

Secrets in CI/CD

Never hardcode **✗**

Use:

- GitHub Secrets

- Vault
- K8s Secrets

CI/CD Best Practices (FAANG)

- ✓ Fast pipeline
- ✓ Fail fast
- ✓ Small commits
- ✓ Separate build & deploy
- ✓ Rollback strategy
- ✓ Blue-Green / Canary deploy

Deployment Strategies

- Rolling Update
- Blue-Green
- Canary
- Feature Flags

Interview Must-Know Q&A

Q: CI vs CD?

 Integration vs Delivery/Deployment

Q: Why CI/CD important?

 Speed + quality + automation

Q: Where tests run?

 CI pipeline

Q: CI/CD in microservices?

👉 Service-wise pipeline

Real-World Example (Next.js)

Push → Lint → Test → Build

→ Dockerize

→ Deploy to K8s

→ Auto scale

One-Line Summary

CI = merge safely

CD = ship fast

CI/CD = automation + confidence